Introduction to Database Management Systems

Detailed Notes

---

1 Introduction to Databases

1.1 Definition of Database

A **database** is an organized collection of structured data that is stored electronically in a computer system. It is designed to efficiently store, retrieve, manage, and update information. A database is typically controlled by a Database Management System (DBMS), which acts as an interface between the database and its users or application programs.

In simpler terms, a database is like a digital filing cabinet where information is stored in an organized manner so that it can be easily accessed, managed, and updated whenever needed.

1.2 Purpose of Databases

The main purposes of databases are:

Efficient Data Storage: Databases provide a centralized location to store large amounts of data in an organized manner, eliminating the need for multiple scattered files.

Easy Data Retrieval: Users can quickly search and retrieve specific information from the database using queries, without having to manually search through files.

Data Sharing: Multiple users can access the same database simultaneously, enabling collaborative work and information sharing across departments or organizations.

Data Integrity and Consistency: Databases enforce rules and constraints to ensure that the data remains accurate, consistent, and reliable over time.

Data Security: Databases provide mechanisms to control who can access, modify, or delete data, protecting sensitive information from unauthorized access.

Backup and Recovery: Databases offer built-in features for backing up data and recovering it in case of system failures or data loss.

1.3 Real-life Examples and Applications

Databases are used extensively in various domains of everyday life:

Banking Systems: Banks use databases to maintain customer account information, transaction records, loan details, and credit card information. When you withdraw money from an ATM or check your balance online, you're interacting with a database.

E-commerce Websites: Online shopping platforms like Amazon and Flipkart use databases to store product catalogs, customer profiles, order histories, payment information, and inventory data.

Social Media Platforms: Facebook, Instagram, and Twitter use massive databases to store user profiles, posts, comments, likes, friend connections, and multimedia content.

Hospital Management Systems: Hospitals maintain databases containing patient records,

medical histories, appointment schedules, doctor information, and billing details.

Educational Institutions: Schools and universities use databases to manage student records, course information, examination results, attendance, and faculty details.

Library Management Systems: Libraries use databases to catalog books, track issued and returned books, maintain member information, and manage reservations.

Airline Reservation Systems: Airlines use databases to manage flight schedules, seat availability, passenger bookings, ticket pricing, and loyalty programs.

1.4 Data vs. Information

Data refers to raw, unprocessed facts and figures that have no meaning by themselves. Data consists of basic elements like numbers, text, dates, or symbols that are collected and stored but not yet organized or interpreted. For example, "25", "John", "Mumbai" are individual data items.

Information is processed, organized, and structured data that has meaning and context. When data is analyzed, interpreted, and presented in a meaningful way, it becomes information that can be used for decision-making. For example, "John is 25 years old and lives in Mumbai" is information derived from organizing the data.

Key Difference: Data is the raw material, while information is the finished product. Data becomes information when it is given context and purpose.

Example: Consider the numbers: 98, 85, 92, 88. These are just data. But when we say "Student Rahul scored 98, 85, 92, and 88 in his four semester exams, with an average of 90.75%", this becomes meaningful information.

2 File System vs. Database System

2.1 File System

A **file system** is a traditional method of storing data where information is kept in separate files, typically managed by the operating system. Each file contains data in a specific format, and application programs directly access these files to read or write data.

In a file system, each department or application might maintain its own set of files, leading to isolated data storage with limited sharing capabilities.

2.2 Database System

A **database system** consists of a database (the actual data storage) and a Database Management System (DBMS) that manages the data. It provides a centralized, integrated approach to data storage where all data is stored in a structured format and accessed through the DBMS rather than directly by applications.

In a database system, data is shared across multiple applications and users, with the DBMS handling all data operations, security, and integrity.

2.3 Limitations of Traditional File Systems

Data Redundancy: The same data is often duplicated across multiple files maintained by different departments or applications. For example, a customer's address might be stored separately in sales files, billing files, and shipping files. This leads to unnecessary duplication and waste of storage space.

Data Inconsistency: When the same data exists in multiple places, updating it in one location doesn't automatically update it everywhere. This leads to inconsistent data. For instance, if a customer changes their address and it's updated in the sales file but not in the billing file, the records become inconsistent.

Difficulty in Data Access: Retrieving specific information requires writing new programs or modifying existing ones. There's no easy way to perform ad-hoc queries. If management wants a new report format, programmers must write custom code to extract and format the data.

Data Isolation: Since data is scattered across various files in different formats, it becomes difficult to write programs that access data from multiple files. Integration of data from different sources is challenging.

Integrity Problems: Maintaining data accuracy and validity is difficult in file systems. Consistency constraints (like "account balance should not be negative") must be coded into every program that accesses the file, leading to potential errors and inconsistencies.

Atomicity Problems: File systems lack built-in mechanisms to ensure that operations either complete fully or not at all. For example, in a money transfer, if the system crashes after deducting money from one account but before adding it to another, the data becomes inconsistent with no automatic recovery.

Concurrent Access Anomalies: When multiple users try to access and modify the same file simultaneously, file systems provide limited coordination, potentially leading to data corruption or loss of updates.

Security Problems: File systems offer limited security features. It's difficult to enforce access restrictions at a granular level. Typically, a user either has complete access to a file or no access at all, with no middle ground for partial access based on specific data items.

2.4 Advantages of DBMS

Data Integrity: A DBMS enforces integrity constraints and rules to ensure data accuracy and consistency. For example, it can ensure that age values are positive numbers, email addresses have valid formats, and foreign key references point to existing records. These constraints are defined once in the database and automatically enforced for all operations.

Data Security: DBMS provides sophisticated security mechanisms including user authentication, authorization, and access control. Different users can be granted different levels of access (read, write, delete) to different parts of the database. Sensitive data can be encrypted, and audit trails can track who accessed or modified what data and when.

Redundancy Reduction: In a DBMS, data is stored in a centralized manner with minimal duplication. Information that would be repeated across multiple files in a file system is stored only once in the database and referenced when needed. This not only saves storage space but also eliminates update anomalies.

Data Consistency: Since data is stored centrally with minimal redundancy, updates to data are made in one place and automatically reflected wherever that data is used. This maintains

consistency across the entire system.

Data Sharing: Multiple users and applications can access the same database simultaneously. The DBMS manages concurrent access and ensures that users don't interfere with each other's operations.

Easy Data Access: DBMS provides query languages (like SQL) that allow users to retrieve and manipulate data easily without writing complex programs. Ad-hoc queries can be executed interactively to get any desired information.

Backup and Recovery: DBMS includes built-in mechanisms for regular data backups and recovery procedures. In case of system failures, the database can be restored to a consistent state, preventing data loss.

Data Independence: The structure of the database can be changed without affecting the application programs that use it, providing flexibility in database design and evolution.

3 Relational Databases

3.1 Definition and Functions

A **relational database** is a type of database that organizes data into one or more tables (also called relations) consisting of rows and columns. The relationships between different tables are established through common fields, typically using keys.

The relational database was proposed by E.F. Codd in 1970 and has since become the most widely used database model due to its simplicity, flexibility, and powerful querying capabilities.

Main Functions of Relational Databases:

Data Organization: Data is structured into tables with clearly defined columns (attributes) and rows (records), making it easy to understand and manage.

Data Retrieval: Relational databases support SQL (Structured Query Language) for querying and retrieving data based on various conditions and criteria.

Relationship Management: Tables can be related to each other through keys, allowing complex relationships to be represented and queried efficiently.

Data Manipulation: Users can insert new data, update existing data, and delete unwanted data using standardized SQL commands.

Data Integrity: Relational databases enforce various constraints (primary key, foreign key, unique, not null) to maintain data accuracy and consistency.

3.2 Relational Model Basics

The relational model represents data as a collection of relations (tables). Each relation consists of a set of tuples (rows) with each tuple having the same set of attributes (columns).

Key Concepts:

Relation/Table: A table is the basic structure in a relational database. It consists of rows and columns. Each table represents an entity (like Student, Course, Employee) and stores all data about that entity.

Tuple/Row/Record: A row in a table represents a single instance or record of the entity. For example, in a Student table, each row would represent one specific student with all their

details.

Attribute/Column/Field: A column in a table represents a specific property or characteristic of the entity. For example, in a Student table, columns might include StudentID, Name, Age, Email, etc.

Domain: The domain is the set of allowed values for an attribute. For example, the domain for Age might be positive integers between 1 and 120, while the domain for Gender might be {'Male', 'Female', 'Other'}.

Degree: The degree of a relation is the number of attributes (columns) it contains. A table with 5 columns has degree 5.

Cardinality: The cardinality of a relation is the number of tuples (rows) it contains. A table with 100 students has cardinality 100.

Schema: The schema defines the structure of a table, including its name, the names of its attributes, and the data types of those attributes. It's like the blueprint of the table.

3.3 Tables, Rows, and Columns

Tables: A table is a collection of related data organized in rows and columns. Each table has a unique name and stores data about a specific entity. For example, a STUDENT table stores information about students, a COURSE table stores information about courses.

Tables are also called relations because they represent a mathematical relation between attributes. The intersection of a row and column contains a single data value.

Rows (Tuples): A row represents a single, complete record in the table. Each row contains values for all the columns defined in the table. Rows are also called tuples or records.

For example, in a STUDENT table, one row might be:

```
101 | Rahul Kumar | 20 | rahul@email.com | Computer Science
```

This represents one student with StudentID 101, name Rahul Kumar, age 20, email rahul@email.com, and department Computer Science.

Columns (Attributes): A column represents a specific property or characteristic that all records in the table share. Each column has a name and a data type. Columns are also called attributes or fields.

For example, a STUDENT table might have columns: StudentID (integer), Name (string), Age (integer), Email (string), Department (string).

All values in a particular column must be of the same data type and represent the same kind of information.

3.4 Keys: Primary Key and Foreign Key

Keys are special attributes (or combinations of attributes) that serve to uniquely identify records and establish relationships between tables.

Primary Key:

A **primary key** is an attribute (or a combination of attributes) that uniquely identifies each row in a table. No two rows can have the same primary key value, and primary key values cannot be NULL (empty).

The primary key is the most important key in a table as it ensures that each record can be uniquely identified and accessed. When you need to refer to a specific record, you use its primary key value.

Properties of Primary Key:

- **Uniqueness:** Each value must be unique across all rows
- **Not NULL:** Cannot contain NULL or empty values
- **Minimal:** Should contain the minimum number of attributes necessary for uniqueness
- **Stable:** Values should not change over time

Example: In a STUDENT table, StudentID would typically be the primary key because each student has a unique ID. Name cannot be a primary key because multiple students might have the same name.

Foreign Key:

A **foreign key** is an attribute (or combination of attributes) in one table that refers to the primary key of another table. It establishes a relationship between two tables by linking them together.

The foreign key creates a parent-child relationship where the table containing the primary key is the parent table, and the table containing the foreign key is the child table. The foreign key value in the child table must either be NULL or match a primary key value in the parent table.

Purpose of Foreign Key:

- Establishes relationships between tables
- Maintains referential integrity (ensures that relationships remain consistent)
- Prevents orphan records (child records without a corresponding parent)

Example: Consider two tables:

- STUDENT table with StudentID (primary key), Name, Age
- ENROLLMENT table with EnrollmentID (primary key), StudentID (foreign key), CourseID, Grade

Here, StudentID in the ENROLLMENT table is a foreign key that references StudentID (primary key) in the STUDENT table. This means every enrollment record must belong to a student that exists in the STUDENT table.

4 Basic SQL Syntax

SQL (Structured Query Language) is the standard language for interacting with relational databases. It allows users to create, modify, query, and manage databases.

SQL commands are divided into different categories based on their functionality:

4.1 DDL (Data Definition Language)

DDL commands are used to define and modify the structure of database objects like tables, indexes, and schemas. These commands affect the database structure, not the data itself.

CREATE Command:

The CREATE command is used to create new database objects, most commonly tables. When creating a table, you specify the table name, column names, data types, and constraints.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    ...
);
```

Example:

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT,
    Email VARCHAR(100) UNIQUE,
    Department VARCHAR(50)
);
```

This creates a STUDENT table with five columns. StudentID is the primary key (unique and not null), Name cannot be NULL, Email must be unique, and Age and Department can be NULL.

DROP Command:

The DROP command is used to permanently delete database objects like tables. Once a table is dropped, all its data and structure are completely removed and cannot be recovered (unless you have a backup).

Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE Student;
```

This permanently deletes the STUDENT table along with all its data. Use this command with extreme caution as it's irreversible.

ALTER Command:

The ALTER command is used to modify the structure of an existing table. You can add new columns, delete existing columns, modify column definitions, or add/remove constraints.

Syntax for adding a column:

```
ALTER TABLE table_name
ADD column_name datatype;
```

Syntax for deleting a column:

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Syntax for modifying a column:

```
ALTER TABLE table_name
MODIFY COLUMN column_name new_datatype;
```

Examples:

```
-- Add a new Phone column
ALTER TABLE Student
ADD Phone VARCHAR(15);

-- Remove the Age column
ALTER TABLE Student
DROP COLUMN Age;

-- Change Name column to allow 100 characters
ALTER TABLE Student
MODIFY COLUMN Name VARCHAR(100);
```

4.2 DML (Data Manipulation Language)

DML commands are used to manipulate the data stored in tables. These commands affect the data itself, not the table structure.

SELECT Command:

The SELECT command is used to retrieve data from one or more tables. It's the most frequently used SQL command and allows you to query the database to get specific information.

Basic Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Examples:

```
-- Select all columns from Student table
SELECT * FROM Student;

-- Select specific columns
SELECT Name, Age FROM Student;

-- Select with condition
```

```
SELECT Name, Email
FROM Student
WHERE Department = 'Computer Science';

-- Select with multiple conditions
SELECT Name, Age
FROM Student
WHERE Age > 20 AND Department = 'Electronics';
```

The WHERE clause is optional and is used to filter records based on specified conditions. The asterisk (*) means "all columns".

INSERT Command:

The INSERT command is used to add new rows (records) into a table. You can insert a complete row by providing values for all columns, or insert partial data by specifying which columns to fill.

Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Examples:

```
-- Insert a complete row
INSERT INTO Student (StudentID, Name, Age, Email, Department)
VALUES (101, 'Rahul Kumar', 20, 'rahul@email.com',
        'Computer Science');

-- Insert partial data (other columns will be NULL)
INSERT INTO Student (StudentID, Name, Department)
VALUES (102, 'Priya Singh', 'Electronics');

-- Insert multiple rows at once
INSERT INTO Student (StudentID, Name, Age, Department)
VALUES
    (103, 'Amit Sharma', 21, 'Mechanical'),
    (104, 'Sneha Patel', 19, 'Civil'),
    (105, 'Rajesh Kumar', 22, 'Computer Science');
```

UPDATE Command:

The UPDATE command is used to modify existing data in a table. You specify which columns to update and what their new values should be, along with a condition to identify which rows to update.

Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Examples:

```
-- Update a single column for one student
UPDATE Student
SET Age = 21
WHERE StudentID = 101;

-- Update multiple columns
UPDATE Student
SET Email = 'new.email@example.com', Department = 'IT'
WHERE StudentID = 102;

-- Update all rows matching a condition
UPDATE Student
SET Department = 'CSE'
WHERE Department = 'Computer Science';
```

Important: Always use a WHERE clause with UPDATE. If you omit the WHERE clause, ALL rows in the table will be updated, which is usually not what you want.

DELETE Command:

The DELETE command is used to remove rows from a table. You specify which rows to delete using a WHERE clause.

Syntax:

```
DELETE FROM table_name
WHERE condition;
```

Examples:

```
-- Delete a specific student
DELETE FROM Student
WHERE StudentID = 101;

-- Delete all students from a specific department
DELETE FROM Student
WHERE Department = 'Mechanical';

-- Delete students older than 25
DELETE FROM Student
WHERE Age > 25;

-- Delete all rows (use with extreme caution!)
DELETE FROM Student;
```

Important: Like UPDATE, always use a WHERE clause with DELETE unless you intentionally want to delete all rows. DELETE removes only the data, not the table structure (use DROP TABLE for that).

End of Notes
Best of luck for your exam!